GSoC 2015 Proposal - Lluís Gómez i Bigordà

lgomez@cvc.uab.es
27/03/2015

Improve and expand Scene Text Detection module in OpenCV

This is a proposal for the improvement and expansion of the work done during the last two GSoC editions in scene text detection and recognition. In the following I list the proposed tasks for this year GsoC and then I describe the implementation details for each of the listed tasks.

GSoC2015 Proposal Tasks

T1. Convolutional Neural Networks for Scene Text Recognition.
T2. Efficient Text-likeliness Pre-filters.
T3. Algorithmic improvements of the current pipeline.
T4. Evaluation for standard datasets (ICDAR RRC).

The main proposal for GSoC2015 consists basically in T1 and T2. Tasks T3 and T4 were already included in previous year proposal, but we did not had time to finish them. They are included as optional tasks that may be done if there is enough time and interest.

GSoC2015 Implementation Details

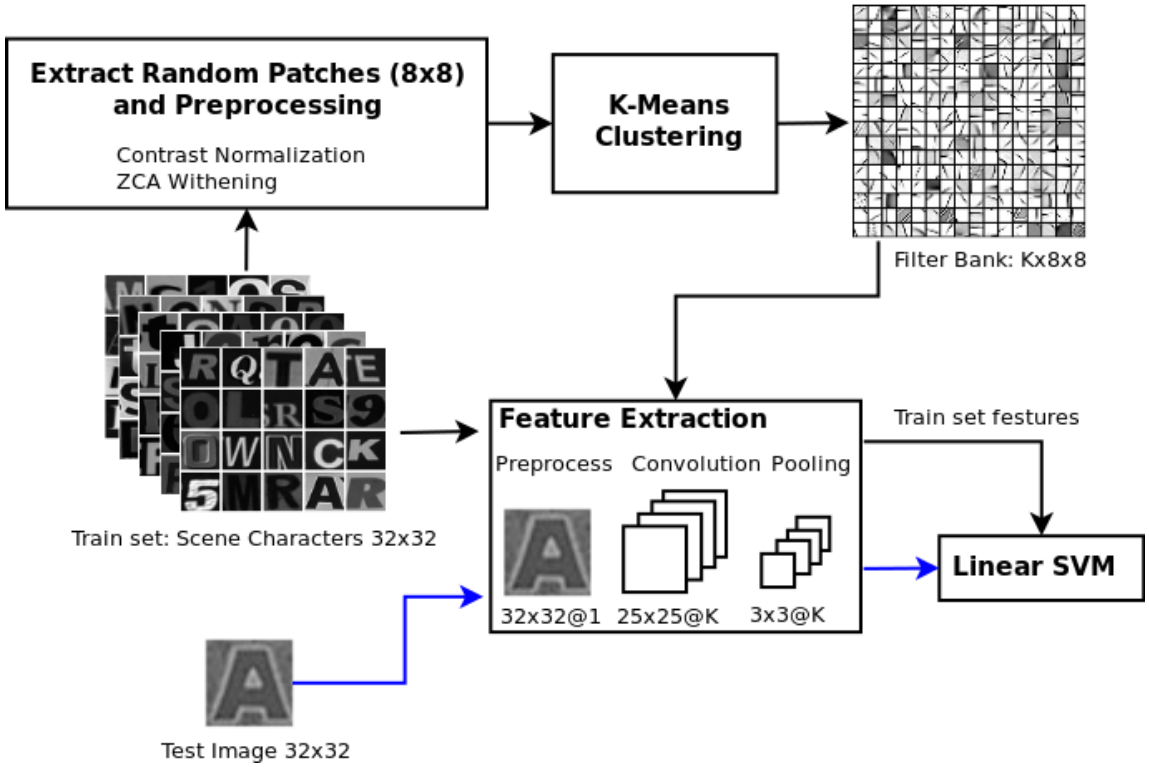
T1. Convolutional Neural Networks for Scene Text Recognition.

Motivation: Currently implemented text recognition methods in OpenCV (**cv::text::OCRTesseract** and **cv::text::OCRHMMDecoder**) are based in binarization. Under certain conditions (e.g. well focussed and bi-level text) such a binarization-based approach provide state-of-the-art recognition, but in many other cases a proper binarization of text components is not feasible, and such approaches fail.

Proposal: Convolutional Neural Networks are nowadays de-facto standard for object detection/recognition tasks in natural scene images, scene text recognition is not an exception [Jaderberg 2014]. While implementation of complex deep networks [Jaderberg 2014] is out of the scope of this project, it turns out that there exist proposals of simpler designs [Coates2011][Wang2012] that can be realistically implemented and may produce recognition rates near state-of-the-art.

T1.1 Single Layer Networks for Character Recognition

The use of single layer convolutional nets has been proposed for character and digits recognition in scene images in [Coates2011] and [Netzer2011] respectively. The main advantages of this method are its relatively easy implementation, and small training requirements. The following diagram summarizes the different modules to be implemented:



As can be appreciated many of the modules can be built using the OpenCV existing functionality. E.g. cv::eigen and cv::calcCovarMatrix for pre-processing, cv::kmeans for the filter bank creation, cv::filter2D for the convolution, … and the linear SVM can be adopted from BSD licensed LibLinear library.

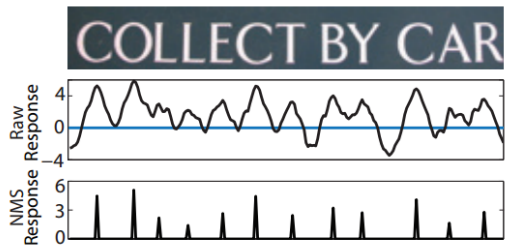
A first implementation of the above method yield a 80% accuracy rate in the ICDAR character recognition dataset.

The task includes building pre-trained classifiers for characters (26/62 classes), and digits (10 classes). Training data is available both from real images (~15000 samples) and synthetic data (~50000 samples).

T1.2 Beam Search

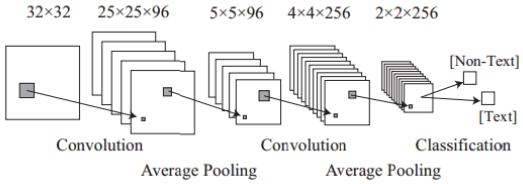
Individual character recognition described in **T1.1** could be easily integrated with the existing class cv::text::OCRHMMDecoder to provide word level recognition using a language model. However in the existing pipeline this requires to previously segment words and character candidates, and this is not trivial. The idea in this task is to

implement the Beam Search algorithm in order recognize best candidate words from an over-segmentation. Given a text line candidate, the character classifier described in **T1.1** is applied in a horizontal sliding window, producing a characterness response over the line. With a Non Maximal/Minimal Suppression the best character candidates locations, as well as space candidate locations, are identified and form an over-segmentation. The following Figure from [Wang2012] illustrates the idea:



The Beam Search algorithm, in combination with a language model (as in **cv::text::OCRHMMDecoder**), applied to the text line over-segmentation produces the final recognition result. This algorithm is known to work especially well in the case where a lexicon (a list of tens to hundreds of candidate words) is provided for each particular image, as in the well known SVT[Wang2011] dataset.
In order to implement the sliding window classifier in an efficient way the idea is to perform first the convolutions of the whole text line candidate, and then use a pooling movable window to extract the features of **T1.1**.

T1.3 Multiple-Layer Networks
This task extends the feature extractor of **T1.1** by stacking two layers of convolution as in [Wang2012]:



In [Wang2012] the filters in the first layer are learned with K-Means (as in **T1.1**) and kept fixed, while the parameters in the second layer are fine-tuned by backpropagation of classification error.
My proposal is to train second layer parameters with an external software (e.g. Theano, or Caffe) and then export them in XML format to be used at test time in OpenCV. The idea requires to build a very simple design for the second layer, so the input/output is exactly the same in OpenCV and the external software. Using a soft-max classifier seems feasible and easy to implement both in OpenCV and in the external CNN software. Moreover, since the number of trainable parameters is not very large the amount of required training data/time is perfectly manageable. Using

this architecture the accuracy of the character classifier is expected to increase between 2 to 5 points.

T2. Efficient Text-likeliness Pre-filters.

Motivation: The existing pipeline in the **cv::text** module relies in the traditional scheme of Detection/Segmentation(binarization)/Recognition, where each of the pipeline modules tries to maximize their individual performance. Thus, the detection algorithms are designed to maximize F-score even if detection recall is not perfect. Another possible approach, by looking at the end-to-end problem as a whole, is to rely more in the final Recognizer decisions (if it is good enough), and allows the detection module to be less restrictive but providing higher recall rates. The problem with this design in the current OpenCV text module is that **cv::text::OCRTesseract** is quite slow. Thus, the motivation of this task is to provide efficient text-likeliness filter algorithms that can evaluate a confidence value for a given bounding box in a very fast way, and to use this measure to filter (or to rank) the candidates.

Proposal: There are a number of simple heuristics that can be implemented straightforward: e.g. counting the number of regions in a candidate bounding box, thresholding their sizes, etc. Another possible option is to analyze fast texture features like the edge-based stroke features in [Neumann2013]. Since all these algorithms need a notion of bounding box to work on, the proposal is to apply them in two different contexts: 1) as an optional filter to the currently implemented detection methods, and 2) in a new “Object Proposals” framework.

T2.1 Text likeliness pre-filters

- **Region Counts consistency**: This is an heuristic filter that can be implemented straightforward as a set of thresholds to the number/sizes of the MSER regions in a given bounding box. An alternative approach could be the use of an efficient data structure to measure the number of regions that exist in a box. This can be done by accumulating the count of bottom-right corners of all MSER regions in the image in an integral representation, allowing the computation of similar objectness measures as in the EdgeBoxes method [Zitnick2014] in O(1). E.g. the number of regions that exist in the box minus those that overlap the box's boundary has already been used in [Jaderberg2014] as a text box proposal method. The intuition is that the feature combining this measure for a box and for a concentric smaller sibling box may be a very good indicator for text likeliness.
- **Stroke features**: Text likeliness can be measured by analyzing the concentration of areas with strong edges and/or the variance of segments’ lengths between pixels with opposite edge orientations. In [Neumann2013] strokes are efficiently detected by convolving the image gradient with a set of oriented bar filters at different scales. This algorithm is simple and can be implemented in OpenCV.

- **Normed Gradients**: Following the approach of the BING paper [Cheng2014] text likeliness can be measured as the presence of groups of objects with well-defined closed boundary by looking at the norm of gradients of individual regions in a very efficient way (a few milliseconds).
- **Integral HOG/LBP**: HOG, LBP, or other texture based features that can be accumulated in integral images are also interesting alternatives to explore.

T2.2 Text Object Proposals

The filters implemented in task T2.1 need a notion of box in which to be applied. The normal operation would be to apply them to the output of the existing detection algorithms in the **cv::text** module. However, if their implementation allows computing the text likeliness measures in constant time complexity, it make sense to exploit them in other ways. For example, using a sliding window as in [Zitnick2014] [Cheng2014]. Another option is to use the hierarchical clustering already implemented in **cv::text::erGrouping** as a tree of text candidates and evaluate their responses at every node of the hierarchy.

T2.3 Character likeliness pre-filters

While text likeliness measures in T2.1 are designed for text part groups (i.e. words or lines), it is also possible to use them in the context of character likeliness. In such scenario, we can think in using those filters as an extension of the existing **cv::text::ERFilter** algorithm, both adding them in to the available classifier cascade, or creating a new/different character detector.

T3. Algorithmic improvements of the current pipeline.

Two proposals of simple algorithmic improvements of the current pipeline in order to reduce their time complexity:

T3.1 Reducing the runtime of the cv::text::ERFilter algorithm

Most of the time taken by the algorithm is wasted in repeated calls to the **eval()** method of the AdaBoost classifier. This task consist in implementing simple heuristic filters before calling **classifier->eval()** : i.e. similarly as we do with the minArea and maxArea filters but with the other features, we can evaluate a few simple conditions before doing the call. Thresholds for this simple filters can be easily learned from the training set of character regions we already have.

T3.2 Reducing the runtime of the cv::text::erGrouping algorithm

The hierarchical clustering has a quadratic complexity on the number of regions. Runtime can be reduced by analyzing consecutive overlapping splits of the regions' set by their sizes/intensity-levels. This procedure does not affect detection performance if the splits are well designed, and may speed up the algorithm in a factor of x4.

T4. Evaluation for standard datasets (ICDAR RRC).

Task T4 consist in pushing code to the new **cv::datasets** module implementing load and benchmarking of the ICDAR 2015 Robust Reading Datasets: Born-Digital Image Text, Scene Text, and Incidental Scene Text.

References

Coates, A., Carpenter, B., Case, C., Satheesh, S., Suresh, B., Wang, T., ... & Ng, A. Y. (2011, September). Text detection and character recognition in scene images with unsupervised feature learning. In *Document Analysis and Recognition (ICDAR), 2011 International Conference on* (pp. 440-445). IEEE.

Wang, T., Wu, D. J., Coates, A., & Ng, A. Y. (2012, November). End-to-end text recognition with convolutional neural networks. In *Pattern Recognition (ICPR), 2012 21st International Conference on* (pp. 3304-3308). IEEE.

Netzer, Y., Wang, T., Coates, A., Bissacco, A., Wu, B., & Ng, A. Y. (2011). Reading digits in natural images with unsupervised feature learning. In *NIPS workshop on deep learning and unsupervised feature learning* (Vol. 2011, No. 2, p. 5). Granada, Spain.

Jaderberg, M., Simonyan, K., Vedaldi, A., & Zisserman, A. (2014). Reading Text in the Wild with Convolutional Neural Networks. *arXiv preprint arXiv:1412.1842*.

Wang, K., Babenko, B., & Belongie, S. (2011, November). End-to-end scene text recognition. In *Computer Vision (ICCV), 2011 IEEE International Conference on*(pp. 1457-1464). IEEE.

Zitnick, C. L., & Dollár, P. (2014). Edge boxes: Locating object proposals from edges. In *Computer Vision–ECCV 2014* (pp. 391-405). Springer International Publishing.

Neumann, L., & Matas, J. (2013, December). Scene text localization and recognition with oriented stroke detection. In *Computer Vision (ICCV), 2013 IEEE International Conference on* (pp. 97-104). IEEE.

Cheng, M. M., Zhang, Z., Lin, W. Y., & Torr, P. (2014, June). BING: Binarized normed gradients for objectness estimation at 300fps. In *Computer Vision and Pattern Recognition (CVPR), 2014 IEEE Conference on* (pp. 3286-3293). IEEE.